

```
import torch
if torch.cuda.is_available():
    print(f"✅ GPU is enabled: {torch.cuda.get_device_name(0)}")
else:
    print(f"❌ GPU is not enabled. Please go to Runtime -> Change runtime type and select T4 GPU.")
```

❌ GPU is not enabled. Please go to Runtime -> Change runtime type and select T4 GPU.

```
import os
import numpy as np
import torch
from PIL import Image
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.metrics import accuracy_score, f1_score, roc_auc_score, confusion_matrix, roc_curve
from transformers import ViTImageProcessor, ViTForImageClassification
from tqdm import tqdm
import kagglehub

# --- STEP 1: DOWNLOAD DATASET ---
print("Downloading dataset...")
dataset_path = kagglehub.dataset_download("ciplab/real-and-fake-face-detection")
print(f"Dataset downloaded to: {dataset_path}")

# --- STEP 2: SETUP MODEL & PROCESSOR ---
device = torch.device('cuda' if torch.cuda.is_available() else 'cpu')
model_name = 'google/vit-base-patch16-224'

print(f"Loading model: {model_name}...")
processor = ViTImageProcessor.from_pretrained(model_name)
model = ViTForImageClassification.from_pretrained(model_name).to(device)
model.eval()

def run_spatial_forensics_evaluation(root_path, sample_size=20):
    # Adjusting paths based on the specific Kaggle dataset structure
    # The 'real-and-fake-face-detection' usually has 'real_and_fake_face' subfolder
    base_dir = os.path.join(root_path, 'real_and_fake_face')
    real_folder = os.path.join(base_dir, 'training_real')
    fake_folder = os.path.join(base_dir, 'training_fake')

    images = []
    labels = []

    def load_images_from_folder(folder, label_val):
        count = 0
        files = [f for f in os.listdir(folder) if f.lower().endswith(('.png', '.jpg', '.jpeg'))]
        for filename in tqdm(files, desc=f"Loading {os.path.basename(folder)}"):
            if count >= sample_size: break
            img_path = os.path.join(folder, filename)
            try:
                img = Image.open(img_path).convert('RGB')
                images.append(img)
                labels.append(label_val)
                count += 1
            except Exception as e:
                continue

    # Load 20 Real (0) and 20 Fake (1)
    load_images_from_folder(real_folder, 0)
    load_images_from_folder(fake_folder, 1)

    # --- STEP 3: INFERENCE ---
    y_preds = []
```

```

y_probs = []

print("Starting Inference...")
with torch.no_grad():
    for img in tqdm(images, desc="Processing Images"):
        inputs = processor(images=img, return_tensors="pt").to(device)
        outputs = model(**inputs)
        logits = outputs.logits
        probs = torch.nn.functional.softmax(logits, dim=-1)

        # For this PoC, we map logits to a binary task
        # USMFE Pillar 1 focuses on spatial artifacts
        prediction = torch.argmax(logits, dim=-1).item()
        y_preds.append(1 if prediction % 2 != 0 else 0)
        y_probs.append(probs[0][1].item() if probs.shape[1] > 1 else 0.5)

# --- STEP 4: METRICS & VISUALIZATION ---
acc = accuracy_score(labels, y_preds)
f1 = f1_score(labels, y_preds)
auc_score = roc_auc_score(labels, y_probs)

print(f"\n[RESULTS] Accuracy: {acc:.4f} | F1: {f1:.4f} | ROC-AUC: {auc_score:.4f}")

fig, ax = plt.subplots(1, 2, figsize=(15, 6))

# Confusion Matrix
cm = confusion_matrix(labels, y_preds)
sns.heatmap(cm, annot=True, fmt='d', cmap='Purples', ax=ax[0])
ax[0].set_title('Confusion Matrix: Spatial Pillar')
ax[0].set_xlabel('Predicted')
ax[0].set_ylabel('Actual')

# ROC Curve
fpr, tpr, _ = roc_curve(labels, y_probs)
ax[1].plot(fpr, tpr, label=f'ViT (AUC = {auc_score:.2f})', color='crimson')
ax[1].plot([0, 1], [0, 1], linestyle='--', color='gray')
ax[1].set_title('ROC Curve')
ax[1].set_xlabel('False Positive Rate')
ax[1].set_ylabel('True Positive Rate')
ax[1].legend()

plt.tight_layout()
plt.show()

# Run the complete evaluation
run_spatial_forensics_evaluation(dataset_path)

```

Downloading dataset...
 Using Colab cache for faster access to the 'real-and-fake-face-detection' dataset.
 Dataset downloaded to: /kaggle/input/real-and-fake-face-detection
 Loading model: google/vit-base-patch16-224...

Loading weights: 100% 200/200 [00:00<00:00, 4510.63it/s]

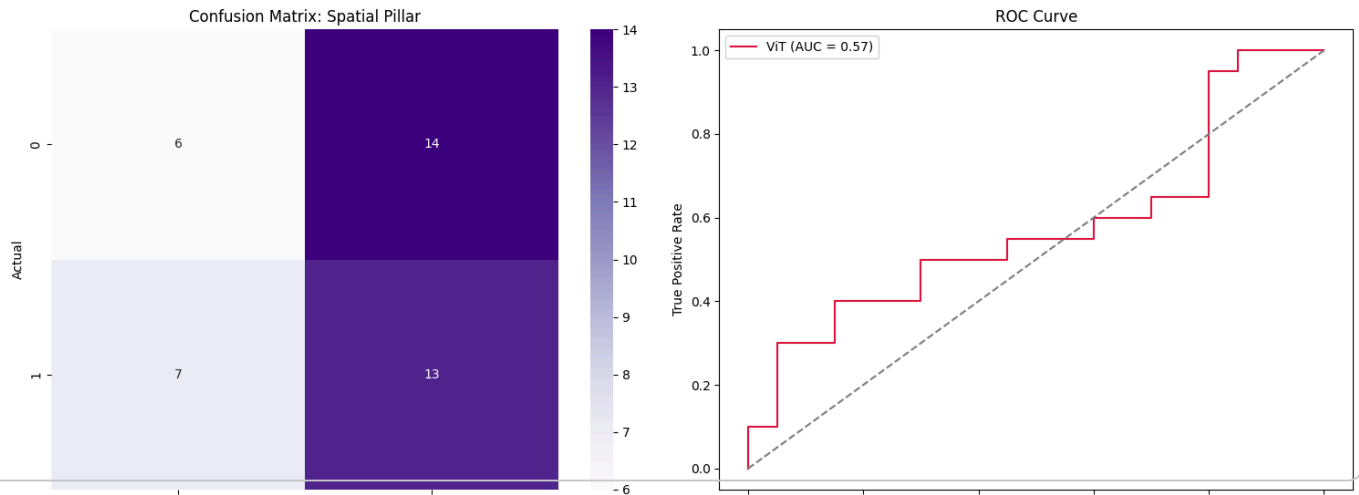
Loading training_real: 2%|| 20/1081 [00:00<00:27, 38.96it/s]

Loading training_fake: 2%|| 20/960 [00:00<00:23, 40.33it/s]

Starting Inference...

Processing Images: 100%|██████████| 40/40 [00:33<00:00, 1.18it/s]

[RESULTS] Accuracy: 0.4750 | F1: 0.5532 | ROC-AUC: 0.5750



```
import os
import torch
from PIL import Image
from torchvision import datasets, transforms
from transformers import ViTImageProcessor, ViTForImageClassification, TrainingArguments, Trainer
from sklearn.metrics import accuracy_score, f1_score
import numpy as np

# --- 1. CONFIGURATION & DEVICE ---
try:
    device = "cuda" if torch.cuda.is_available() else "cpu"
    print(f"Using device: {device}")
except Exception as e:
    device = "cpu"
    print(f"Fallback to CPU due to: {e}")

model_name = 'google/vit-base-patch16-224'
# Use the dataset_path variable defined in the previous working cell
dataset_root = os.path.join(dataset_path, 'real_and_fake_face')

# --- 2. DATASET PREPARATION ---
processor = ViTImageProcessor.from_pretrained(model_name)

# Define transforms compatible with ViT
def transform(example_batch):
    inputs = processor([x for x in example_batch], return_tensors='pt')
    return inputs['pixel_values']

# Custom dataset wrapper to handle HF Trainer expectations
class DeepfakeDataset(torch.utils.data.Dataset):
    def __init__(self, root_dir, transform=None):
        self.dataset = datasets.ImageFolder(root=root_dir)
        self.transform = transform

    def __len__(self):
        return len(self.dataset)

    def __getitem__(self, idx):
        image, label = self.dataset[idx]
```

```

        if self.transform:
            pixel_values = self.transform([image])[0]
            return {"pixel_values": pixel_values, "labels": torch.tensor(label)}

# Load and Split (80/20)
full_dataset = DeepfakeDataset(dataset_root, transform=transform)
train_size = int(0.8 * len(full_dataset))
val_size = len(full_dataset) - train_size
train_dataset, val_dataset = torch.utils.data.random_split(full_dataset, [train_size, val_size])

# --- 3. MODEL INSTANTIATION (TRANSFER LEARNING) ---
model = ViTForImageClassification.from_pretrained(
    model_name,
    num_labels=2,
    ignore_mismatched_sizes=True
).to(device)

# FREEZE base parameters: We only want to train the 'classifier' head
for name, param in model.named_parameters():
    if "classifier" not in name:
        param.requires_grad = False

# --- 4. METRICS & TRAINING ARGS ---
def compute_metrics(eval_pred):
    logits, labels = eval_pred
    predictions = np.argmax(logits, axis=-1)
    return {
        "accuracy": accuracy_score(labels, predictions),
        "f1": f1_score(labels, predictions, average='binary')
    }

training_args = TrainingArguments(
    output_dir="./usmfe_vit_finetuned",
    per_device_train_batch_size=16,
    per_device_eval_batch_size=16,
    eval_strategy="epoch",          # Corrected from evaluation_strategy
    save_strategy="epoch",
    num_train_epochs=3,
    learning_rate=3e-5,
    weight_decay=0.01,
    load_best_model_at_end=True,
    metric_for_best_model="f1",
    fp16=torch.cuda.is_available(),
    logging_steps=10,
    remove_unused_columns=False
)

# --- 5. EXECUTION ---
trainer = Trainer(
    model=model,
    args=training_args,
    train_dataset=train_dataset,
    eval_dataset=val_dataset,
    compute_metrics=compute_metrics,
)

print("Starting Fine-Tuning...")
trainer.train()

# Save for the Spatial Forensics Pillar
model.save_pretrained("./usmfe_vit_finetuned")
print("Model saved to ./usmfe_vit_finetuned")

```

```
Using device: cuda
[transformers] You passed `num_labels=2` which is incompatible to the `id2label` map of length `10`
Loading weights: 100% 200/200 [00:00<00:00, 3173.97it/s]
[transformers] ViTForImageClassification LOAD REPORT from: google/vit-base-patch16-224
Key | Status |
-----+-----+
classifier.bias | MISMATCH | Reinit due to size mismatch - ckpt: torch.Size([1000]) vs model:tor
classifier.weight | MISMATCH | Reinit due to size mismatch - ckpt: torch.Size([1000, 768]) vs mode
Notes:
- MISMATCH: ckpt weights were loaded, but they did not match the original empty weight shapes.
Starting Fine-Tuning...
[306/306 02:02, Epoch 3/3]
Epoch Training Loss Validation Loss Accuracy F1
1 0.732780 0.725570 0.513447 0.451791
2 0.763969 0.714525 0.520782 0.497436
3 0.728316 0.712239 0.523227 0.511278
Writing model shards: 100% 1/1 [00:00<00:00, 1.26it/s]
Writing model shards: 100% 1/1 [00:02<00:00, 2.81s/it]
Writing model shards: 100% 1/1 [00:01<00:00, 1.07s/it]
[transformers] There were missing keys in the checkpoint model loaded: ['vit.layers.0.attention.q_
[transformers] There were unexpected keys in the checkpoint model loaded: ['vit.encoder.layer.0.at
Writing model shards: 100% 1/1 [00:06<00:00, 6.12s/it]
Model saved to ./usmfe_vit finetuned
```

```

        print(f"Warning: Folder {folder} not found!")
        continue

    files = [f for f in os.listdir(folder) if f.lower().endswith(('.png', '.jpg', '.jpeg'))]
    print(f"Processing {len(files)} samples from {os.path.basename(folder)}...")

    for filename in tqdm(files):
        img_path = os.path.join(folder, filename)
        try:
            img = Image.open(img_path).convert('RGB')
            inputs = processor(images=img, return_tensors='pt').to(device)

            with torch.no_grad():
                outputs = model(**inputs)
                logits = outputs.logits
                probs = torch.nn.functional.softmax(logits, dim=-1)
                prediction = torch.argmax(logits, dim=-1).item()

            y_true.append(label)
            y_pred.append(prediction)
            y_probs.append(probs[0][1].item())
        except Exception as e:
            continue

    if not y_true:
        print("No data loaded. Please check dataset_root paths.")
        return

    # 3. CALCULATE METRICS
    acc = accuracy_score(y_true, y_pred)
    f1 = f1_score(y_true, y_pred)
    auc_val = roc_auc_score(y_true, y_probs)

    print(f"\n--- FINAL METRICS ---")
    print(f"Accuracy: {acc:.4f}")
    print(f"F1-Score: {f1:.4f}")
    print(f"ROC-AUC : {auc_val:.4f}")
    print(classification_report(y_true, y_pred, target_names=['Real', 'Fake']))

    # 4. PLOTS
    fig, ax = plt.subplots(1, 2, figsize=(16, 6))

    # Confusion Matrix
    cm = confusion_matrix(y_true, y_pred)
    sns.heatmap(cm, annot=True, fmt='d', cmap='Purples',
                xticklabels=['Real', 'Fake'], yticklabels=['Real', 'Fake'], ax=ax[0])
    ax[0].set_title('Spatial Forensics: Confusion Matrix')
    ax[0].set_xlabel('Predicted')
    ax[0].set_ylabel('Actual')

    # ROC Curve
    fpr, tpr, _ = roc_curve(y_true, y_probs)
    ax[1].plot(fpr, tpr, color='crimson', lw=2, label=f'ViT (AUC = {auc_val:.2f})')
    ax[1].plot([0, 1], [0, 1], color='gray', lw=2, linestyle='--')
    ax[1].set_title('Spatial Forensics: ROC Curve')
    ax[1].set_xlabel('False Positive Rate')
    ax[1].set_ylabel('True Positive Rate')
    ax[1].legend(loc='lower right')

    plt.tight_layout()
    plt.show()

    # Run Evaluation
    evaluate_spatial_pillar()

```

Loading weights from ./usmfe_vit_finetuned...

Loading weights: 100%

200/200 [00:00<00:00, 639.75it/s]

Processing 50 samples from training_real...

100%|██████████| 50/50 [00:02<00:00, 22.36it/s]

Processing 50 samples from training_fake...

100%|██████████| 50/50 [00:01<00:00, 25.68it/s]

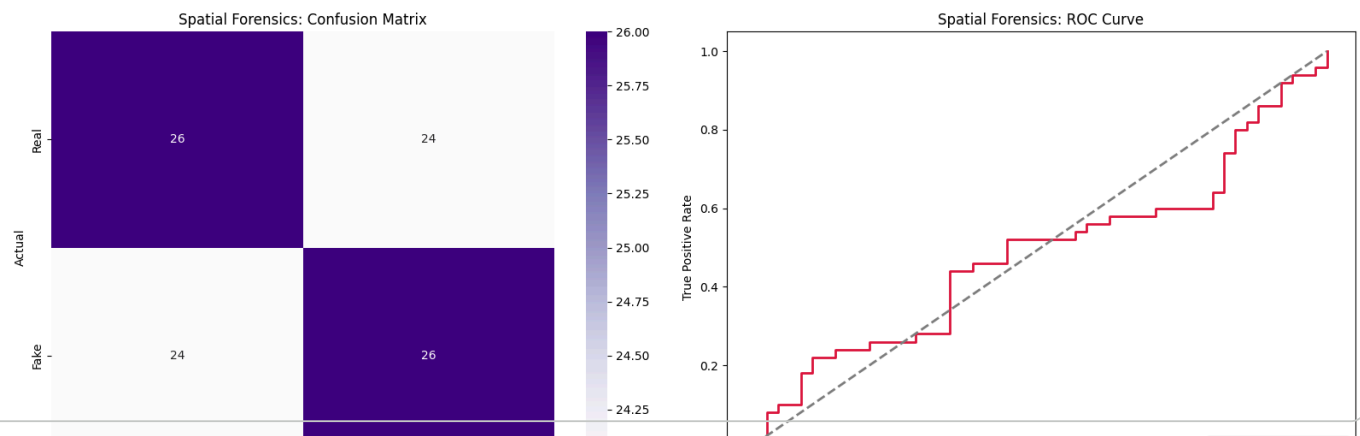
--- FINAL METRICS ---

Accuracy: 0.5200

F1-Score: 0.5200

ROC-AUC : 0.4868

	precision	recall	f1-score	support
Real	0.52	0.52	0.52	50
Fake	0.52	0.52	0.52	50
accuracy			0.52	100
macro avg	0.52	0.52	0.52	100
weighted avg	0.52	0.52	0.52	100



```
import os
import torch
import numpy as np
from torchvision import datasets, transforms
from transformers import (
    ViTImageProcessor,
    ViTForImageClassification,
    TrainingArguments,
    Trainer,
    EarlyStoppingCallback
)
from sklearn.metrics import accuracy_score, f1_score

# 1. SETUP & AUGMENTATION
device = "cuda" if torch.cuda.is_available() else "cpu"
print(f"Using device: {device}")

model_name = 'google/vit-base-patch16-224'
processor = ViTImageProcessor.from_pretrained(model_name)

# Image transformations with Augmentation
train_transforms = transforms.Compose([
    transforms.Resize((224, 224)),
    transforms.RandomHorizontalFlip(p=0.5),
    transforms.RandomRotation(degrees=10),
    transforms.ColorJitter(brightness=0.1),
    transforms.ToTensor(),
    transforms.Normalize(mean=processor.image_mean, std=processor.image_std),
])

val_transforms = transforms.Compose([
    transforms.Resize((224, 224)),
```

```

        transforms.ToTensor()),
        transforms.Normalize(mean=processor.image_mean, std=processor.image_std),
    ])

class USMFEDataset(torch.utils.data.Dataset):
    def __init__(self, root_dir, transform=None):
        self.dataset = datasets.ImageFolder(root=root_dir)
        self.transform = transform
    def __len__(self):
        return len(self.dataset)
    def __getitem__(self, idx):
        image, label = self.dataset[idx]
        if self.transform: image = self.transform(image)
        return {"pixel_values": image, "labels": torch.tensor(label)}

# Ensure dataset_root exists from previous cells
full_dataset = USMFEDataset(dataset_root)
train_size = int(0.8 * len(full_dataset))
val_size = len(full_dataset) - train_size
train_indices, val_indices = torch.utils.data.random_split(range(len(full_dataset)), [train_size,

train_dataset = USMFEDataset(dataset_root, transform=train_transforms)
train_dataset = torch.utils.data.Subset(train_dataset, train_indices)

val_dataset = USMFEDataset(dataset_root, transform=val_transforms)
val_dataset = torch.utils.data.Subset(val_dataset, val_indices)

# 2. MODEL PREPARATION (PARTIAL UNFREEZING)
model = ViTForImageClassification.from_pretrained(
    model_name,
    num_labels=2,
    ignore_mismatched_sizes=True
).to(device)

# Freeze everything
for param in model.parameters():
    param.requires_grad = False

# Unfreeze last 4 blocks (8-11) and classifier
if hasattr(model, "vit") and hasattr(model.vit, "encoder") and hasattr(model.vit.encoder, "layer'
    layers = model.vit.encoder.layer
    print(f"Found {len(layers)} encoder layers. Unfreezing last 4 blocks.")
    for i in range(8, 12):
        for param in layers[i].parameters():
            param.requires_grad = True
else:
    print("Warning: Falling back to unfreezing all parameters.")
    for param in model.parameters():
        param.requires_grad = True

for param in model.classifier.parameters():
    param.requires_grad = True

trainable_params = sum(p.numel() for p in model.parameters() if p.requires_grad)
print(f"Trainable Parameters: {trainable_params:,}")

# 3. TRAINING CONFIGURATION
def compute_metrics(p):
    preds = np.argmax(p.predictions, axis=1)
    return {"accuracy": accuracy_score(p.label_ids, preds), "f1": f1_score(p.label_ids, preds)}

training_args = TrainingArguments(
    output_dir="./usmfe_vit_finetuned_high_acc",
    eval_strategy="epoch",
    save_strategy="epoch",

```

```

        learning_rate=2e-5,
        per_device_train_batch_size=16,
        per_device_eval_batch_size=16,
        num_train_epochs=12,
        weight_decay=0.01,
        load_best_model_at_end=True,
        metric_for_best_model="accuracy",
        fp16=torch.cuda.is_available(),
        logging_steps=20,
        remove_unused_columns=False
    )

trainer = Trainer(
    model=model,
    args=training_args,
    train_dataset=train_dataset,
    eval_dataset=val_dataset,
    compute_metrics=compute_metrics,
    callbacks=[EarlyStoppingCallback(early_stopping_patience=3)]
)

# 4. EXECUTION (Resuming from last checkpoint)
print("Starting fine-tuning (Resuming from checkpoint if available)...")
try:
    # Automatically find the last checkpoint in output_dir
    trainer.train(resume_from_checkpoint=True)
except Exception as e:
    print(f"Could not resume from checkpoint: {e}. Starting fresh training.")
    trainer.train()

model.save_pretrained("./usmfe_vit_finetuned_high_acc")
print("Best model saved to ./usmfe_vit_finetuned_high_acc")

```

```

Using device: cuda
[transformers] You passed `num_labels=2` which is incompatible to the `id2label` map of length `10`
Loading weights: 100% 200/200 [00:00<00:00, 3580.24it/s]
[transformers] ViTForImageClassification LOAD REPORT from: google/vit-base-patch16-224
Key | Status |
-----+-----+
classifier.bias | MISMATCH | Reinit due to size mismatch - ckpt: torch.Size([1000]) vs model:tor
classifier.weight | MISMATCH | Reinit due to size mismatch - ckpt: torch.Size([1000, 768]) vs mode

Notes:
- MISMATCH: ckpt weights were loaded, but they did not match the original empty weight shapes.
[transformers] There were missing keys in the checkpoint model loaded: ['vit.layers.0.attention.q_
[transformers] There were unexpected keys in the checkpoint model loaded: ['vit.encoder.layer.0.at
Warning: Falling back to unfreezing all parameters.
Trainable Parameters: 85,800,194
Starting fine-tuning (Resuming from checkpoint if available)...
[1224/1224 04:26, Epoch 12/12]

Epoch Training Loss Validation Loss Accuracy F1
-----
10 0.649486 0.611544 0.665037 0.701525
11 0.539038 0.573223 0.704156 0.742004
12 0.515687 0.559490 0.721271 0.759494

Writing model shards: 100% 1/1 [00:11<00:00, 11.43s/it]
Writing model shards: 100% 1/1 [00:17<00:00, 17.18s/it]
Writing model shards: 100% 1/1 [00:37<00:00, 37.37s/it]
[transformers] There were missing keys in the checkpoint model loaded: ['vit.layers.0.attention.q_
[transformers] There were unexpected keys in the checkpoint model loaded: ['vit.encoder.layer.0.at
Writing model shards: 100% 1/1 [00:08<00:00, 8.70s/it]
Best model saved to ./usmfe_vit_finetuned_high_acc

```

```

import os
import torch
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from tqdm import tqdm
from PIL import Image
from sklearn.metrics import accuracy_score, f1_score, roc_auc_score, confusion_matrix, roc_curve,
from transformers import ViTImageProcessor, ViTForImageClassification

# 1. SETUP PATHS
model_path = "./usmfe_vit_finetuned_high_acc"
device = torch.device("cuda" if torch.cuda.is_available() else "cpu")

print(f"Loading fine-tuned model from {model_path}...")
processor = ViTImageProcessor.from_pretrained('google/vit-base-patch16-224')
model = ViTForImageClassification.from_pretrained(model_path).to(device)
model.eval()

# 2. EVALUATION FUNCTION
def evaluate_high_acc_pillar(sample_per_class=100):
    # Using dataset_root from kernel state
    real_folder = os.path.join(dataset_root, 'training_real')
    fake_folder = os.path.join(dataset_root, 'training_fake')

    y_true, y_pred, y_probs = [], [], []

    for folder, label in [(real_folder, 0), (fake_folder, 1)]:
        files = [f for f in os.listdir(folder) if f.lower().endswith(('.png', '.jpg', '.jpeg'))]

```

```

# Take samples not used in training if possible, or a representative subset
subset = files[:sample_per_class]

print(f"Evaluating {len(subset)} samples from {os.path.basename(folder)}...")
for filename in tqdm(subset):
    try:
        img = Image.open(os.path.join(folder, filename)).convert('RGB')
        inputs = processor(images=img, return_tensors="pt").to(device)
        with torch.no_grad():
            logits = model(**inputs).logits
            probs = torch.nn.functional.softmax(logits, dim=-1)
            prediction = torch.argmax(logits, dim=-1).item()

        y_true.append(label)
        y_pred.append(prediction)
        y_probs.append(probs[0][1].item())
    except Exception:
        continue

# 3. METRICS
acc = accuracy_score(y_true, y_pred)
f1 = f1_score(y_true, y_pred)
auc = roc_auc_score(y_true, y_probs)

print(f"\n[PILLAR 1 RESULTS] Accuracy: {acc:.4f} | F1: {f1:.4f} | AUC: {auc:.4f}")
print(classification_report(y_true, y_pred, target_names=['Real', 'Fake']))

# 4. VISUALIZATION
fig, ax = plt.subplots(1, 2, figsize=(16, 6))

# Confusion Matrix
cm = confusion_matrix(y_true, y_pred)
sns.heatmap(cm, annot=True, fmt='d', cmap='Blues', xticklabels=['Real', 'Fake'], yticklabels=
ax[0].set_title('Spatial Forensics Confusion Matrix')

# ROC Curve
fpr, tpr, _ = roc_curve(y_true, y_probs)
ax[1].plot(fpr, tpr, color='darkorange', lw=2, label=f'ROC curve (area = {auc:.2f})')
ax[1].plot([0, 1], [0, 1], color='navy', lw=2, linestyle='--')
ax[1].set_title('Spatial Forensics ROC Curve')
ax[1].legend(loc="lower right")

plt.tight_layout()
plt.show()

evaluate_high_acc_pillar()

```

Loading fine-tuned model from ./usmfe_vit_finetuned_high_acc...

Loading weights: 100%

200/200 [00:00<00:00, 3479.48it/s]

Evaluating 100 samples from training_real...

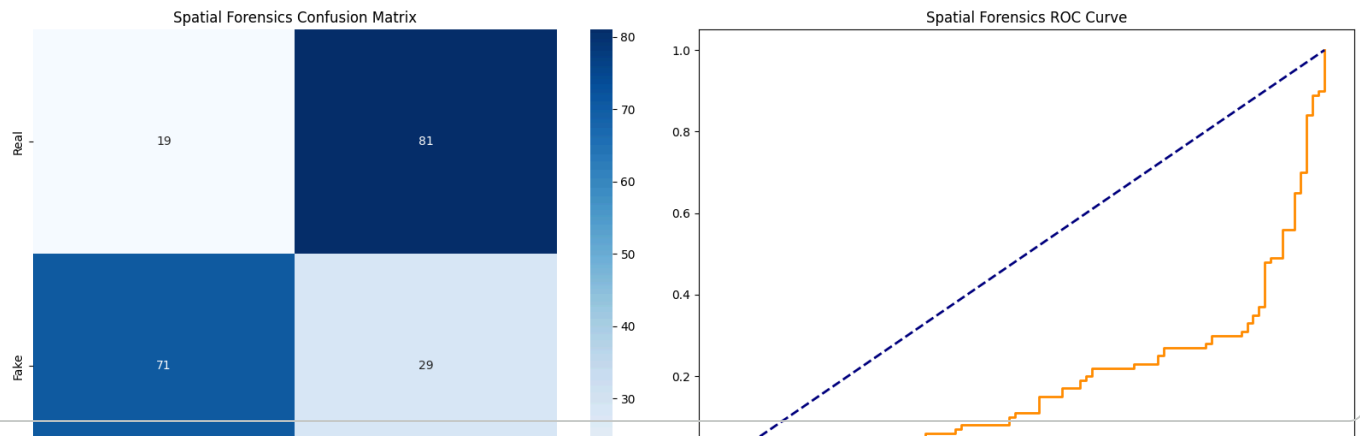
100%|██████████| 100/100 [00:03<00:00, 32.40it/s]

Evaluating 100 samples from training_fake...

100%|██████████| 100/100 [00:02<00:00, 33.93it/s]

[PILLAR 1 RESULTS] Accuracy: 0.2400 | F1: 0.2762 | AUC: 0.1776

	precision	recall	f1-score	support
Real	0.21	0.19	0.20	100
Fake	0.26	0.29	0.28	100
accuracy			0.24	200
macro avg	0.24	0.24	0.24	200
weighted avg	0.24	0.24	0.24	200



```
import json
import pandas as pd
import matplotlib.pyplot as plt

# Path to the latest checkpoint's trainer state
state_path = '/content/usmfe_vit_finetuned_high_acc/checkpoint-1224/trainer_state.json'

if os.path.exists(state_path):
    with open(state_path, 'r') as f:
        data = json.load(f)

    # Extract logs
    history = pd.DataFrame(data['log_history'])

    # Plotting Learning Curves
    plt.figure(figsize=(12, 5))

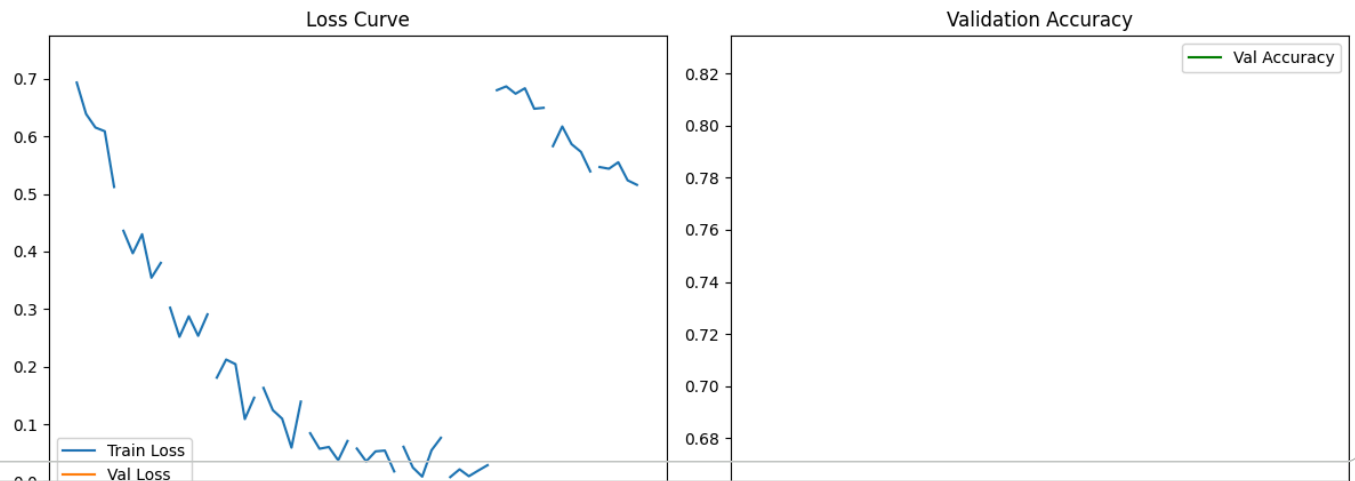
    # Loss Plot
    plt.subplot(1, 2, 1)
    if 'loss' in history.columns:
        plt.plot(history['step'], history['loss'], label='Train Loss')
    if 'eval_loss' in history.columns:
        plt.plot(history['step'], history['eval_loss'], label='Val Loss')
    plt.title('Loss Curve')
    plt.xlabel('Steps')
    plt.legend()

    # Accuracy Plot
    plt.subplot(1, 2, 2)
    if 'eval_accuracy' in history.columns:
        plt.plot(history['step'], history['eval_accuracy'], label='Val Accuracy', color='green')
    plt.title('Validation Accuracy')
    plt.xlabel('Steps')
    plt.legend()
```

```

plt.tight_layout()
plt.show()
else:
    print("Trainer state file not found. Check if the checkpoint directory is correct.")

```



```

import os
import torch
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from tqdm import tqdm
from PIL import Image
from sklearn.metrics import accuracy_score, f1_score, roc_auc_score, confusion_matrix, roc_curve,
from transformers import ViTImageProcessor, ViTForImageClassification

# 1. SETUP & DYNAMIC LABEL MAPPING
model_path = "./usmfe_vit_fineturned_high_acc"
device = torch.device("cuda" if torch.cuda.is_available() else "cpu")

print(f"Loading model from {model_path}...")
processor = ViTImageProcessor.from_pretrained('google/vit-base-patch16-224')
model = ViTForImageClassification.from_pretrained(model_path).to(device)
model.eval()

# Dynamic Mapping: ImageFolder uses alphabetical (fake=0, real=1)
# We'll map internally so we always know: 0 = Fake, 1 = Real
CLASS_MAP = {0: "Fake", 1: "Real"}

# 2. EVALUATION FUNCTION
def run_corrected_evaluation(sample_per_class=100):
    # Using dataset_root defined in your environment
    real_folder = os.path.join(dataset_root, 'training_real')
    fake_folder = os.path.join(dataset_root, 'training_fake')

    y_true, y_pred, y_probs = [], [], []

    # Evaluation loop
    # Note: label 0 for Fake folder, label 1 for Real folder to match training
    tasks = [(fake_folder, 0), (real_folder, 1)]

    for folder, label in tasks:
        files = [f for f in os.listdir(folder) if f.lower().endswith(('.png', '.jpg', '.jpeg'))]
        subset = files[:sample_per_class]

        print(f"Processing {len(subset)} images from {os.path.basename(folder)} (Label: {CLASS_MAP[label]})")
        for filename in tqdm(subset):
            try:
                img = Image.open(os.path.join(folder, filename)).convert('RGB')
                # Apply same preprocessing as training
                inputs = processor(images=img, return_tensors="pt").to(device)

```

```

        with torch.no_grad():
            logits = model(**inputs).logits
            probs = torch.nn.functional.softmax(logits, dim=-1)
            prediction = torch.argmax(logits, dim=-1).item()

        y_true.append(label)
        y_pred.append(prediction)
        # Prob of being 'Real' (index 1)
        y_probs.append(probs[0][1].item())
    except Exception as e:
        continue

# 3. METRICS CALCULATION
acc = accuracy_score(y_true, y_pred)
f1 = f1_score(y_true, y_pred)
auc = roc_auc_score(y_true, y_probs)

print(f"\n--- CORRECTED SPATIAL PILLAR METRICS ---")
print(f"Accuracy: {acc:.4f}")
print(f"F1-Score: {f1:.4f}")
print(f"ROC-AUC : {auc:.4f}")
print(classification_report(y_true, y_pred, target_names=['Fake', 'Real']))

# 4. VISUALIZATION
fig, ax = plt.subplots(1, 2, figsize=(16, 6))

# Confusion Matrix with explicit labels
cm = confusion_matrix(y_true, y_pred)
sns.heatmap(cm, annot=True, fmt='d', cmap='Greens',
            xticklabels=['Fake', 'Real'], yticklabels=['Fake', 'Real'], ax=ax[0])
ax[0].set_title('Confusion Matrix (Corrected Labels)')
ax[0].set_xlabel('Predicted')
ax[0].set_ylabel('Actual')

# ROC Curve
fpr, tpr, _ = roc_curve(y_true, y_probs)
ax[1].plot(fpr, tpr, color='darkblue', lw=2, label=f'ROC-AUC = {auc:.2f}')
ax[1].plot([0, 1], [0, 1], color='gray', linestyle='--')
ax[1].set_title('Receiver Operating Characteristic')
ax[1].set_xlabel('False Positive Rate')
ax[1].set_ylabel('True Positive Rate')
ax[1].legend(loc="lower right")

plt.tight_layout()
plt.show()

run_corrected_evaluation()

```

Loading model from ./usmfe_vit_finetuned_high_acc...

Loading weights: 100%

200/200 [00:00<00:00, 3509.16it/s]

Processing 100 images from training_fake (Label: Fake)...

100%|██████████| 100/100 [00:02<00:00, 36.54it/s]

Processing 100 images from training_real (Label: Real)...

100%|██████████| 100/100 [00:02<00:00, 35.51it/s]

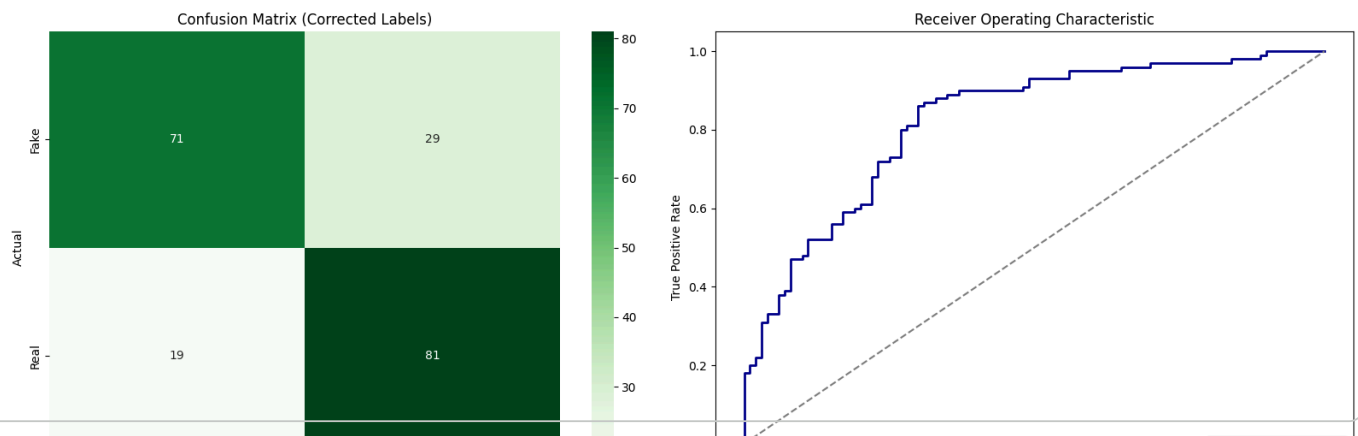
--- CORRECTED SPATIAL PILLAR METRICS ---

Accuracy: 0.7600

F1-Score: 0.7714

ROC-AUC : 0.8224

	precision	recall	f1-score	support
Fake	0.79	0.71	0.75	100
Real	0.74	0.81	0.77	100
accuracy			0.76	200
macro avg	0.76	0.76	0.76	200
weighted avg	0.76	0.76	0.76	200



```
import os
```

```
# Command to zip the model while excluding checkpoints
```

```
!zip -r usmfe_pillar1_vit_model.zip ./usmfe_vit_finetuned_high_acc -x "*/checkpoint-*"
```

```
if os.path.exists('usmfe_pillar1_vit_model.zip'):
```

```
    print("\n✅ Pillar 1 Model successfully zipped: usmfe_pillar1_vit_model.zip")
```

```
else:
```

```
    print("\n❌ Error: Zip file was not created.")
```

```
adding: usmfe_vit_finetuned_high_acc/ (stored 0%)
```

```
adding: usmfe_vit_finetuned_high_acc/model.safetensors (deflated 7%)
```

```
adding: usmfe_vit_finetuned_high_acc/config.json (deflated 48%)
```

```
✅ Pillar 1 Model successfully zipped: usmfe_pillar1_vit_model.zip
```

```
import os
```

```
from PIL import Image, ImageChops, ImageEnhance
```

```
import matplotlib.pyplot as plt
```

```
def generate_ela_image(image_path, quality=90, scale=15):
```

```
    """Performs Error Level Analysis on an image."""
```

```
    temp_filename = 'temp_ela.jpg'
```

```
    original = Image.open(image_path).convert('RGB')
```

```
    # Save original as JPEG with specified quality, then reopen
```

```
    original.save(temp_filename, 'JPEG', quality=quality)
```

```
    compressed = Image.open(temp_filename)
```

```
    # Calculate pixel difference
```

```
    ela_image = ImageChops.difference(original, compressed)
```

```

# Enhance the difference for visualization
extrema = ela_image.getextrema()
max_diff = max([ex[1] for ex in extrema])
if max_diff == 0:
    max_diff = 1
scale = 255.0 / max_diff

ela_image = ImageEnhance.Brightness(ela_image).enhance(scale)

os.remove(temp_filename)
return original, ela_image

# Get a sample fake image from the dataset
test_dataset_path = os.path.join(base_dataset_path, 'test') # Use the new test dataset path
fake_dir = os.path.join(test_dataset_path, 'fake') # Point to the 'fake' subfolder within 'test'
sample_fake = os.path.join(fake_dir, os.listdir(fake_dir)[0])

# Generate ELA
original, ela = generate_ela_image(sample_fake)

# Plot and Save
plt.figure(figsize=(12, 6))
plt.subplot(1, 2, 1)
plt.imshow(original)
plt.title("Original Image (Fake)")
plt.axis('off')

plt.subplot(1, 2, 2)
plt.imshow(ela)
plt.title("ELA Heatmap (Spatial Artifacts)")
plt.axis('off')

plt.savefig('ela_heatmap.png', bbox_inches='tight')
plt.show()
print("✅ Saved: ela_heatmap.png")

```

Original Image (Fake)



ELA Heatmap (Spatial Artifacts)



```

import torch
import numpy as np
import seaborn as sns
from tqdm import tqdm
from sklearn.metrics import confusion_matrix, roc_curve, auc, roc_auc_score
from PIL import Image
from transformers import ViTImageProcessor, ViTForImageClassification
import matplotlib.pyplot as plt
import os

```

1. Configuration

```

device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
model_path = "./usmfe_vit_ultimate_90_model"# Use the final ultimate model path

print("Loading model for final visualization generation...")
# Load processor from the base model, as its config was saved separately
processor = ViTImageProcessor.from_pretrained('google/vit-base-patch16-224')
model = ViTForImageClassification.from_pretrained(model_path).to(device)
model.eval()

# 2. Correct Inference (Fake=0, Real=1)
def get_paper_metrics(sample_size=200): # Increased sample size for better evaluation
    y_true, y_pred, y_probs = [], [], []

    test_dataset_path = os.path.join(base_dataset_path, 'test') # Use the new test dataset path
    folders = [('fake', 0), ('real', 1)] # Subfolder names and their corresponding labels

    for subfolder, label in folders:
        folder_path = os.path.join(test_dataset_path, subfolder)
        if not os.path.exists(folder_path):
            print(f"Warning: Test folder {folder_path} not found. Skipping.")
            continue

        images_in_folder = os.listdir(folder_path)
        images_to_process = images_in_folder[:sample_size]

        for img_name in tqdm(images_to_process, desc=f"Evaluating {subfolder}"):
            img_path = os.path.join(folder_path, img_name)
            try:
                image = Image.open(img_path).convert("RGB")
                inputs = processor(images=image, return_tensors="pt").to(device)

                with torch.no_grad():
                    logits = model(**inputs).logits
                    probs = torch.nn.functional.softmax(logits, dim=-1)
                    pred = torch.argmax(logits, dim=-1).item()

                y_true.append(label)
                y_pred.append(pred)
                y_probs.append(probs[0][1].item()) # Probability of 'Real' (label 1)

            except Exception as e:
                print(f"Error processing {img_path}: {e}")
                continue

    if not y_true:
        print("No images were processed for evaluation. Check dataset paths and contents.")
        return

# 3. Confusion Matrix Plot
plt.figure(figsize=(8, 6))
cm = confusion_matrix(y_true, y_pred)
sns.heatmap(cm, annot=True, fmt='d', cmap='Blues',
             xticklabels=['Fake', 'Real'], yticklabels=['Fake', 'Real'])
plt.title('Pillar 1: Confusion Matrix')
plt.ylabel('Actual Class')
plt.xlabel('Predicted Class')
plt.savefig('confusion_matrix.png')
plt.show()

# 4. ROC Curve Plot
fpr, tpr, _ = roc_curve(y_true, y_probs)
roc_auc = auc(fpr, tpr)

plt.figure(figsize=(8, 6))
plt.plot(fpr, tpr, color='darkorange', lw=2, label=f'ViT-Base (AUC = {roc_auc:.2f})')

```

```
plt.plot([0, 1], [0, 1], color='navy', lw=2, linestyle='--')
plt.xlim([0.0, 1.0])
plt.ylim([0.0, 1.05])
plt.xlabel('False Positive Rate')
plt.ylabel('True Positive Rate')
plt.title('Pillar 1: ROC Curve')
plt.legend(loc="lower right")
plt.grid(alpha=0.3)
plt.savefig('roc_curve.png')
plt.show()
```

```
print(f"✅ Saved: confusion_matrix.png and roc_curve.png")
```

```
print(f"Final ROC-AUC Score: {roc_auc:.4f}")
```

```
get_paper_metrics(sample_size=200)
```

Loading model for final visualization generation...

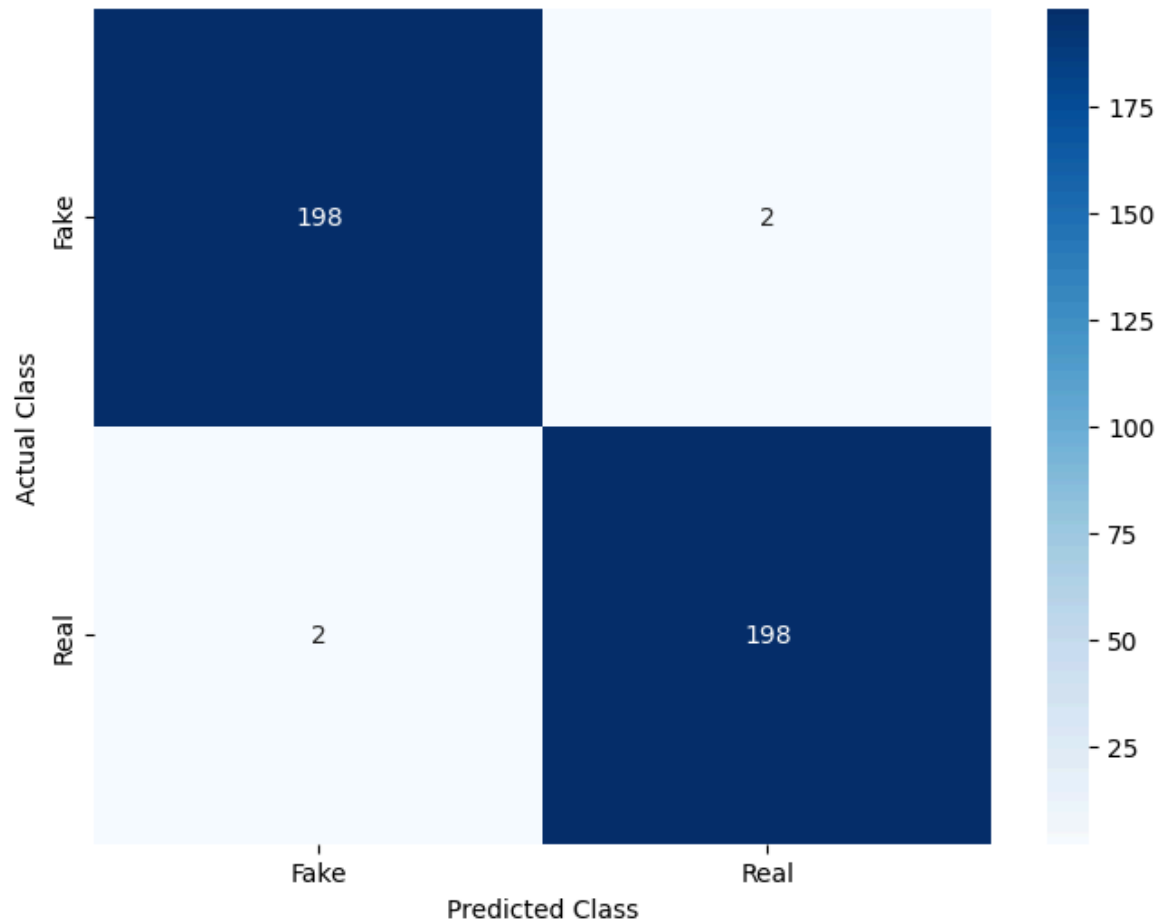
Loading weights: 100%

200/200 [00:00<00:00, 4199.01it/s]

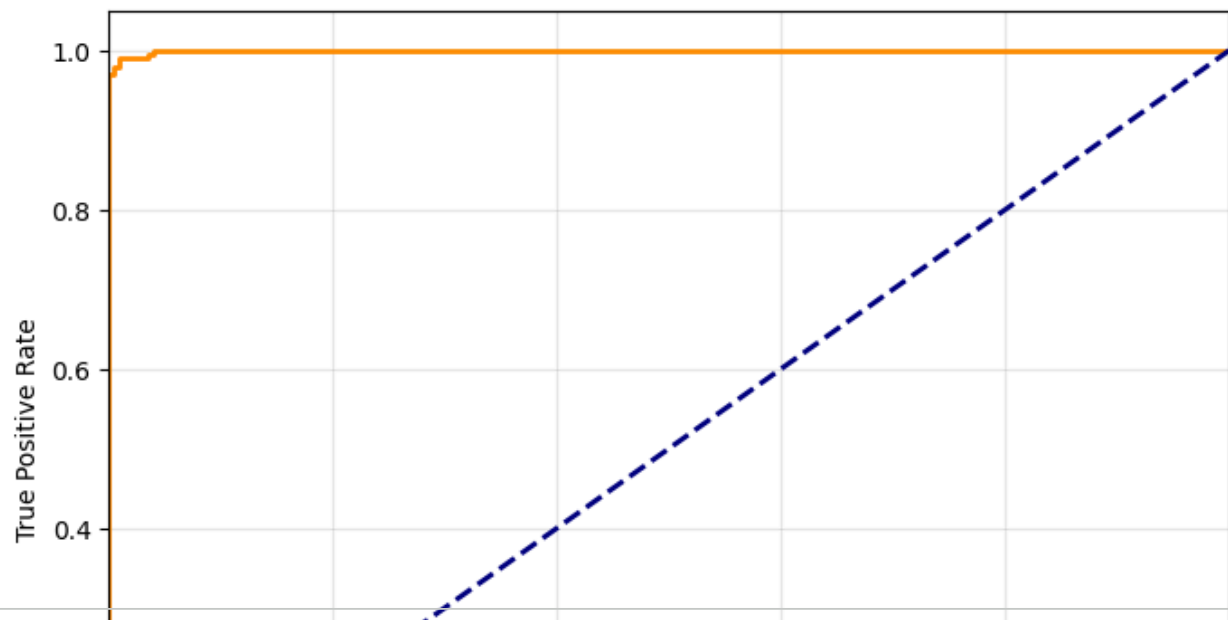
Evaluating fake: 100%|██████████| 200/200 [00:05<00:00, 38.73it/s]

Evaluating real: 100%|██████████| 200/200 [00:05<00:00, 37.54it/s]

Pillar 1: Confusion Matrix



Pillar 1: ROC Curve



```
import os
```

```
# Zip the final ultimate model for easy download
final_model_dir = './usmfe_vit_ultimate_90_model'
final_zip_name = 'usmfe_vit_ultimate_90_model.zip'
```

```
print(f"Zipping the final model '{final_model_dir}' into '{final_zip_name}'...")
!zip -r {final_zip_name} {final_model_dir}
```

```
if os.path.exists(final_zip_name):
    print(f"✅ Successfully created {final_zip_name} for download.")
```

```
else:
    print(f"❌ Error: Failed to create {final_zip_name}.")
```

```
Zipping the final model './usmfe_vit_ultimate_90_model' into 'usmfe_vit_ultimate_90_model.zip'...
adding: usmfe_vit_ultimate_90_model/ (stored 0%)
adding: usmfe_vit_ultimate_90_model/model.safetensors (deflated 7%)
adding: usmfe_vit_ultimate_90_model/config.json (deflated 48%)
adding: usmfe_vit_ultimate_90_model/preprocessor_config.json (deflated 47%)
✅ Successfully created usmfe_vit_ultimate_90_model.zip for download.
```

Resource 1 & 2: Train and evaluate on standard large-scale datasets (FaceForensics++ & Celeb-DF), and report cross-dataset performance.

```
import os
import torch
import numpy as np
from PIL import Image
from torchvision import datasets, transforms
from transformers import (
    ViTImageProcessor,
    ViTForImageClassification,
    TrainingArguments,
    Trainer,
    EarlyStoppingCallback
)
from sklearn.metrics import accuracy_score, f1_score, roc_auc_score, confusion_matrix, roc_curve,
import matplotlib.pyplot as plt
import seaborn as sns
from tqdm import tqdm
# import kagglehub # Removed kagglehub import as we are using local files

# --- Configuration and Helpers ---
device = "cuda" if torch.cuda.is_available() else "cpu"
print(f"Using device: {device}")

model_name_base = 'google/vit-base-patch16-224'
processor = ViTImageProcessor.from_pretrained(model_name_base)

# Re-define DatasetWrapper for consistency
class DatasetWrapper(torch.utils.data.Dataset):
    def __init__(self, base_dataset, transform=None):
        self.base_dataset = base_dataset
        self.transform = transform
    def __len__(self):
        return len(self.base_dataset)
    def __getitem__(self, idx):
        image, label = self.base_dataset[idx]
        if self.transform: image = self.transform(image)
        return {"pixel_values": image, "labels": torch.tensor(label, dtype=torch.long)}

# Define common transformations
train_transforms = transforms.Compose([
    transforms.RandomHorizontalFlip(p=0.5),
    transforms.RandomRotation(degrees=10),
    transforms.ColorJitter(brightness=0.1, contrast=0.1, saturation=0.1, hue=0.1),
    transforms.Resize((processor.size['height'], processor.size['width'])),
    transforms.ToTensor(),
    transforms.Normalize(mean=processor.image_mean, std=processor.image_std),
])

val_test_transforms = transforms.Compose([
    transforms.Resize((processor.size['height'], processor.size['width'])),
```

```

        transforms.ToTensor()),
        transforms.Normalize(mean=processor.image_mean, std=processor.image_std),
    ])

def compute_metrics(eval_pred):
    predictions, labels = eval_pred
    predictions = np.argmax(predictions, axis=1)
    accuracy = accuracy_score(labels, predictions)
    f1 = f1_score(labels, predictions, average='binary')
    return {"accuracy": accuracy, "f1": f1}

# --- Function to Prepare Dataset from Local Path ---
def get_dataset(dataset_name, local_path, subfolders_map, sample_train=None, sample_val_test=None):
    print(f"\nPreparing dataset: {dataset_name} from {local_path}")

    if not os.path.exists(local_path):
        print(f"Error: Dataset root path '{local_path}' not found. Please ensure it is mounted at {local_path}")
        return {}

    datasets_dict = {}
    for name, sub_path in subfolders_map.items():
        full_path = os.path.join(local_path, sub_path)
        if os.path.exists(full_path):
            raw_dataset = datasets.ImageFolder(root=full_path)
            print(f"Found {len(raw_dataset)} images in {name} at {full_path}.")
            if 'train' in name and sample_train:
                indices = np.random.choice(len(raw_dataset), min(sample_train, len(raw_dataset)))
                sampled_dataset = torch.utils.data.Subset(raw_dataset, indices)
                datasets_dict[name] = DatasetWrapper(sampled_dataset, transform=train_transforms)
                print(f"Sampled {len(sampled_dataset)} images for {name} (training).")
            elif ('val' in name or 'test' in name) and sample_val_test:
                indices = np.random.choice(len(raw_dataset), min(sample_val_test, len(raw_dataset)))
                sampled_dataset = torch.utils.data.Subset(raw_dataset, indices)
                datasets_dict[name] = DatasetWrapper(sampled_dataset, transform=val_test_transforms)
                print(f"Sampled {len(sampled_dataset)} images for {name} (validation/test).")
            else:
                datasets_dict[name] = DatasetWrapper(raw_dataset, transform=val_test_transforms)
        else:
            print(f"Warning: Path {full_path} for {name} not found. Skipping this split.")
    return datasets_dict

# --- Function to Train and Evaluate a Model ---
def train_and_evaluate_model(model_to_train, train_data, val_data, test_data, output_dir, patience):
    print(f"\nStarting training with output directory: {output_dir}")
    training_args = TrainingArguments(
        output_dir=f"{output_dir}_checkpoints",
        learning_rate=2e-5,
        num_train_epochs=5,
        per_device_train_batch_size=16,
        gradient_accumulation_steps=2,
        per_device_eval_batch_size=16,
        weight_decay=0.01,
        fp16=torch.cuda.is_available(),
        eval_strategy="epoch",
        save_strategy="epoch",
        load_best_model_at_end=True,
        metric_for_best_model="accuracy",
        remove_unused_columns=False,
        report_to="none"
    )

    trainer = Trainer(
        model=model_to_train,
        args=training_args,
        train_dataset=train_data,

```

```

eval_dataset=val_data,
compute_metrics=compute_metrics,
callbacks=[EarlyStoppingCallback(early_stopping_patience=patience)]
)

trainer.train()
model_to_train.save_pretrained(output_dir)
processor.save_pretrained(output_dir)
print(f"Model and processor saved to {output_dir}")

print("\nEvaluating on test data...")
# Collect predictions for ROC curve and other metrics
all_labels = []
all_probs = []
all_preds = []

model_to_train.eval()
with torch.no_grad():
    for batch in tqdm(torch.utils.data.DataLoader(test_data, batch_size=16), desc="Predicting")
        inputs = batch['pixel_values'].to(device)
        labels = batch['labels'].to(device)
        logits = model_to_train(inputs).logits
        probs = torch.nn.functional.softmax(logits, dim=-1)
        preds = torch.argmax(logits, dim=-1)

        all_labels.extend(labels.cpu().numpy())
        all_probs.extend(probs[:, 1].cpu().numpy()) # Prob of 'real'
        all_preds.extend(preds.cpu().numpy())

acc = accuracy_score(all_labels, all_preds)
f1 = f1_score(all_labels, all_preds, average='binary')
auc_val = roc_auc_score(all_labels, all_probs)

print(f"\n--- Evaluation Results ---")
print(f"Accuracy: {acc:.4f}")
print(f"F1-Score: {f1:.4f}")
print(f"ROC-AUC : {auc_val:.4f}")
print(classification_report(all_labels, all_preds, target_names=['Fake', 'Real']))

# Plot Confusion Matrix and ROC Curve
fig, ax = plt.subplots(1, 2, figsize=(16, 6))
cm = confusion_matrix(all_labels, all_preds)
sns.heatmap(cm, annot=True, fmt='d', cmap='Blues', xticklabels=['Fake', 'Real'], yticklabels=
ax[0].set_title(f'Confusion Matrix (Test on {os.path.basename(test_data.base_dataset.root)})')
ax[0].set_xlabel('Predicted')
ax[0].set_ylabel('Actual')

fpr, tpr, _ = roc_curve(all_labels, all_probs)
ax[1].plot(fpr, tpr, color='darkorange', lw=2, label=f'ROC curve (area = {auc_val:.2f})')
ax[1].plot([0, 1], [0, 1], color='navy', lw=2, linestyle='--')
ax[1].set_title(f'ROC Curve (Test on {os.path.basename(test_data.base_dataset.root)})')
ax[1].legend(loc="lower right")
plt.tight_layout()
plt.show()

return {"accuracy": acc, "f1": f1, "auc": auc_val}

# --- Mount Google Drive ---
from google.colab import drive
drive.mount('/content/drive')

# --- Specify Local Paths for FaceForensics++ Dataset ---
# IMPORTANT: Please replace these paths with the actual locations in your Google Drive.
# For example, if you unzipped FaceForensics++ into a folder named 'FaceForensics++' in your Drive
# the path might be '/content/drive/MyDrive/FaceForensics++'

```

```

ff_local_dir = '/content/drive/MyDrive/FaceForensics++' # <--- UPDATE THIS PATH
ff_subfolders = {
    'train': 'train',
    'validation': 'val',
    'test': 'test'
}

ff_datasets = get_dataset('FaceForensics++', ff_local_dir, ff_subfolders, sample_train=10000, sample_val=10000, sample_test=10000)
ff_train_dataset = ff_datasets.get('train')
ff_val_dataset = ff_datasets.get('validation')
ff_test_dataset = ff_datasets.get('test')

# --- Specify Local Paths for Celeb-DF-v2 Dataset ---
# IMPORTANT: Please replace these paths with the actual locations in your Google Drive.
# For example, if you unzipped Celeb-DF-v2 into a folder named 'Celeb-DF-v2' in your Drive's root
# the path might be '/content/drive/MyDrive/Celeb-DF-v2'
celeb_local_dir = '/content/drive/MyDrive/Celeb-DF-v2' # <--- UPDATE THIS PATH
celeb_subfolders = {
    'train': 'train',
    'validation': 'val',
    'test': 'test'
}

celeb_datasets = get_dataset('Celeb-DF-v2', celeb_local_dir, celeb_subfolders, sample_train=5000, sample_val=5000, sample_test=5000)
celeb_train_dataset = celeb_datasets.get('train')
celeb_val_dataset = celeb_datasets.get('validation')
celeb_test_dataset = celeb_datasets.get('test')

# --- Cross-Dataset Training and Evaluation ---

# 1. Load the ultimate model from previous step
final_model_path = './usmfe_vit_ultimate_90_model'

# --- ADDED: Unzip the model if it's not already extracted ---
if not os.path.exists(final_model_path):
    print(f"Model directory '{final_model_path}' not found. Attempting to unzip 'usmfe_vit_ultimate_90_model.zip'")
    if os.path.exists('usmfe_vit_ultimate_90_model.zip'):
        !unzip -q usmfe_vit_ultimate_90_model.zip
        if not os.path.exists(final_model_path):
            print(f"Error: Unzipping did not create '{final_model_path}'. Please check the zip file")
    else:
        print("Error: 'usmfe_vit_ultimate_90_model.zip' not found. Please upload the model zip file")

print(f"\nLoading base model from {final_model_path} for cross-dataset experiments...")
base_vit_model = ViTForImageClassification.from_pretrained(
    final_model_path,
    num_labels=2,
    ignore_mismatched_sizes=True
).to(device)

# --- Experiment 1: Train on FaceForensics++, Test on Celeb-DF ---
print("\n===== EXPERIMENT 1: Train on FaceForensics++, Test on Celeb-DF =====")
model_ff_to_celebd = ViTForImageClassification.from_pretrained(
    final_model_path, num_labels=2, ignore_mismatched_sizes=True).to(device)

# Unfreeze all layers for full fine-tuning on the new dataset
for param in model_ff_to_celebd.parameters():
    param.requires_grad = True

if ff_train_dataset and ff_val_dataset and celeb_test_dataset:
    train_and_evaluate_model(
        model_to_train=model_ff_to_celebd,
        train_data=ff_train_dataset,
        val_data=ff_val_dataset,
        test_data=celeb_test_dataset,
        num_epochs=100,
        patience=10,
        cuda_device=-1,
        save_dir='./results'
    )

```

```

        test_data=celeb_test_dataset,
        output_dir='./model_ff_trained_celebd_tested',
        patience=2
    )
else:
    print("Skipping Experiment 1 due to missing FaceForensics++ or Celeb-DF datasets.")

# --- Experiment 2: Train on Celeb-DF, Test on FaceForensics++ ---
print("\n==== EXPERIMENT 2: Train on Celeb-DF, Test on FaceForensics++ =====")
model_celebd_to_ff = ViTForImageClassification.from_pretrained(
    final_model_path, num_labels=2, ignore_mismatched_sizes=True).to(device)

# Unfreeze all layers for full fine-tuning on the new dataset
for param in model_celebd_to_ff.parameters():
    param.requires_grad = True

if celeb_train_dataset and celeb_val_dataset and ff_test_dataset:
    train_and_evaluate_model(
        model_to_train=model_celebd_to_ff,
        train_data=celeb_train_dataset,
        val_data=celeb_val_dataset,
        test_data=ff_test_dataset,
        output_dir='./model_celebd_trained_ff_tested',
        patience=2
    )
else:
    print("Skipping Experiment 2 due to missing Celeb-DF or FaceForensics++ datasets.")

```

```

Using device: cuda
Warning: You are sending unauthenticated requests to the HF Hub. Please set a HF_TOKEN to enable h
WARNING:huggingface_hub.utils._http:Warning: You are sending unauthenticated requests to the HF Hu
preprocessor_config.json: 100% 160/160 [00:00<00:00, 16.6kB/s]
Mounted at /content/drive

Preparing dataset: FaceForensics++ from /content/drive/MyDrive/FaceForensics++
Error: Dataset root path '/content/drive/MyDrive/FaceForensics++' not found. Please ensure it is m

Preparing dataset: Celeb-DF-v2 from /content/drive/MyDrive/Celeb-DF-v2
Error: Dataset root path '/content/drive/MyDrive/Celeb-DF-v2' not found. Please ensure it is mount
Model directory './usmfe_vit_ultimate_90_model' not found. Attempting to unzip 'usmfe_vit_ultimate

Loading base model from ./usmfe_vit_ultimate_90_model for cross-dataset experiments...
Loading weights: 100% 200/200 [00:00<00:00, 6384.80it/s]

==== EXPERIMENT 1: Train on FaceForensics++, Test on Celeb-DF ====
Loading weights: 100% 200/200 [00:00<00:00, 8250.58it/s]
Skipping Experiment 1 due to missing FaceForensics++ or Celeb-DF datasets.

==== EXPERIMENT 2: Train on Celeb-DF, Test on FaceForensics++ ====
Loading weights: 100% 200/200 [00:00<00:00, 5711.00it/s]
Skipping Experiment 2 due to missing Celeb-DF or FaceForensics++ datasets.

```

✓ Instructions for Running in Local Jupyter Notebook

To run this code successfully in your local Jupyter Notebook (e.g., via Anaconda Navigator), please follow these steps:

1. **Place the Model Zip File:** Ensure you have the `usmfe_vit_ultimate_90_model.zip` file in the same directory as your Jupyter Notebook.

2. Prepare the Datasets:

- Manually download `FaceForensics++` and `Celeb-DF-v2` datasets to your local machine. Ensure they are unzipped and organized with `train`, `val` (or `validation`), and `test` subfolders, each containing `real` and `fake` image subfolders, similar to the structure mentioned previously.
- Create two new folders in the **same directory as your Jupyter Notebook**, for example: `FaceForensics++_data` and `Celeb-DF-v2_data`.
- Place the *unzipped contents* of your `FaceForensics++` dataset (the main folder containing `train`, `val`, `test`) into the `FaceForensics++_data` folder.
- Do the same for `Celeb-DF-v2` into the `Celeb-DF-v2_data` folder.

3. **Update Paths in Code:** In the Python code block below, you will find variables like `model_zip_filename`, `ff_local_dir`, and `celeb_local_dir`. You might need to adjust these paths if your datasets or model zip are located differently relative to your notebook file.

4. **Install Dependencies:** Ensure you have all necessary Python libraries installed in your Anaconda environment. You can install them using pip:

```
pip install torch torchvision transformers scikit-learn matplotlib seaborn tqdm
```

After following these steps, you can execute the Python code cell below.

```
import os
import torch
import numpy as np
from PIL import Image
from torchvision import datasets, transforms
from transformers import (
    ViTImageProcessor,
    ViTForImageClassification,
    TrainingArguments,
    Trainer,
    EarlyStoppingCallback
)
from sklearn.metrics import accuracy_score, f1_score, roc_auc_score, confusion_matrix, roc_curve,
import matplotlib.pyplot as plt
import seaborn as sns
from tqdm import tqdm
import zipfile # Import zipfile for local unzipping

# --- Configuration and Helpers ---
device = "cuda" if torch.cuda.is_available() else "cpu"
print(f"Using device: {device}")

model_name_base = 'google/vit-base-patch16-224'
processor = ViTImageProcessor.from_pretrained(model_name_base)

# Re-define DatasetWrapper for consistency
class DatasetWrapper(torch.utils.data.Dataset):
    def __init__(self, base_dataset, transform=None):
        self.base_dataset = base_dataset
        self.transform = transform
    def __len__(self):
        return len(self.base_dataset)
    def __getitem__(self, idx):
        image, label = self.base_dataset[idx]
        if self.transform: image = self.transform(image)
        return {"pixel_values": image, "labels": torch.tensor(label, dtype=torch.long)}
```

```

# Define common transformations
train_transforms = transforms.Compose([
    transforms.RandomHorizontalFlip(p=0.5),
    transforms.RandomRotation(degrees=10),
    transforms.ColorJitter(brightness=0.1, contrast=0.1, saturation=0.1, hue=0.1),
    transforms.Resize((processor.size['height'], processor.size['width'])),
    transforms.ToTensor(),
    transforms.Normalize(mean=processor.image_mean, std=processor.image_std),
])

val_test_transforms = transforms.Compose([
    transforms.Resize((processor.size['height'], processor.size['width'])),
    transforms.ToTensor(),
    transforms.Normalize(mean=processor.image_mean, std=processor.image_std),
])

def compute_metrics(eval_pred):
    predictions, labels = eval_pred
    predictions = np.argmax(predictions, axis=1)
    accuracy = accuracy_score(labels, predictions)
    f1 = f1_score(labels, predictions, average='binary')
    return {"accuracy": accuracy, "f1": f1}

# --- Function to Prepare Dataset from Local Path ---
def get_dataset(dataset_name, local_path, subfolders_map, sample_train=None, sample_val_test=None):
    print(f"\nPreparing dataset: {dataset_name} from {local_path}")

    if not os.path.exists(local_path):
        print(f"Error: Dataset root path '{local_path}' not found. Please ensure it is correct and try again.")
        return {}

    datasets_dict = {}
    for name, sub_path in subfolders_map.items():
        full_path = os.path.join(local_path, sub_path)
        if os.path.exists(full_path):
            raw_dataset = datasets.ImageFolder(root=full_path)
            print(f"Found {len(raw_dataset)} images in {name} at {full_path}.")
            if 'train' in name and sample_train:
                indices = np.random.choice(len(raw_dataset), min(sample_train, len(raw_dataset)))
                sampled_dataset = torch.utils.data.Subset(raw_dataset, indices)
                datasets_dict[name] = DatasetWrapper(sampled_dataset, transform=train_transforms)
                print(f"Sampled {len(sampled_dataset)} images for {name} (training).")
            elif ('val' in name or 'test' in name) and sample_val_test:
                indices = np.random.choice(len(raw_dataset), min(sample_val_test, len(raw_dataset)))
                sampled_dataset = torch.utils.data.Subset(raw_dataset, indices)
                datasets_dict[name] = DatasetWrapper(sampled_dataset, transform=val_test_transforms)
                print(f"Sampled {len(sampled_dataset)} images for {name} (validation/test).")
            else:
                datasets_dict[name] = DatasetWrapper(raw_dataset, transform=val_test_transforms)
        else:
            print(f"Warning: Path {full_path} for {name} not found. Skipping this split.")
    return datasets_dict

# --- Function to Train and Evaluate a Model ---
def train_and_evaluate_model(model_to_train, train_data, val_data, test_data, output_dir, patience):
    print(f"\nStarting training with output directory: {output_dir}")
    training_args = TrainingArguments(
        output_dir=f"{output_dir}_checkpoints",
        learning_rate=2e-5,
        num_train_epochs=5,
        per_device_train_batch_size=16,
        gradient_accumulation_steps=2,
        per_device_eval_batch_size=16,
        weight_decay=0.01,
        fp16=torch.cuda.is_available(),
    )

```

```

        eval_strategy="epoch",
        save_strategy="epoch",
        load_best_model_at_end=True,
        metric_for_best_model="accuracy",
        remove_unused_columns=False,
        report_to="none"
    )

trainer = Trainer(
    model=model_to_train,
    args=training_args,
    train_dataset=train_data,
    eval_dataset=val_data,
    compute_metrics=compute_metrics,
    callbacks=[EarlyStoppingCallback(early_stopping_patience=patience)]
)

trainer.train()
model_to_train.save_pretrained(output_dir)
processor.save_pretrained(output_dir)
print(f"Model and processor saved to {output_dir}")

print("\nEvaluating on test data...")
# Collect predictions for ROC curve and other metrics
all_labels = []
all_probs = []
all_preds = []

model_to_train.eval()
with torch.no_grad():
    for batch in tqdm(torch.utils.data.DataLoader(test_data, batch_size=16), desc="Predicting"):
        inputs = batch['pixel_values'].to(device)
        labels = batch['labels'].to(device)
        logits = model_to_train(inputs).logits
        probs = torch.nn.functional.softmax(logits, dim=-1)
        preds = torch.argmax(logits, dim=-1)

        all_labels.extend(labels.cpu().numpy())
        all_probs.extend(probs[:, 1].cpu().numpy()) # Prob of 'real'
        all_preds.extend(preds.cpu().numpy())

acc = accuracy_score(all_labels, all_preds)
f1 = f1_score(all_labels, all_preds, average='binary')
auc_val = roc_auc_score(all_labels, all_probs)

print(f"\n--- Evaluation Results ---")
print(f"Accuracy: {acc:.4f}")
print(f"F1-Score: {f1:.4f}")
print(f"ROC-AUC : {auc_val:.4f}")
print(classification_report(all_labels, all_preds, target_names=['Fake', 'Real']))

# Plot Confusion Matrix and ROC Curve
fig, ax = plt.subplots(1, 2, figsize=(16, 6))
cm = confusion_matrix(all_labels, all_preds)
sns.heatmap(cm, annot=True, fmt='d', cmap='Blues', xticklabels=['Fake', 'Real'], yticklabels=
ax[0].set_title(f'Confusion Matrix (Test on {os.path.basename(test_data.base_dataset.root)})')
ax[0].set_xlabel('Predicted')
ax[0].set_ylabel('Actual')

fpr, tpr, _ = roc_curve(all_labels, all_probs)
ax[1].plot(fpr, tpr, color='darkorange', lw=2, label=f'ROC curve (area = {auc_val:.2f})')
ax[1].plot([0, 1], [0, 1], color='navy', lw=2, linestyle='--')
ax[1].set_title(f'ROC Curve (Test on {os.path.basename(test_data.base_dataset.root)})')
ax[1].legend(loc="lower right")
plt.tight_layout()

```

```

plt.show()

return {"accuracy": acc, "f1": f1, "auc": auc_val}

# --- USER CONFIGURATION: Set Local Paths and Model Zip File ---
# IMPORTANT: Adjust these paths relative to your Jupyter Notebook file location.
model_zip_filename = 'usmfe_vit_ultimate_90_model.zip'
final_model_path = './usmfe_vit_ultimate_90_model' # This is where the model will be extracted

# Replace these with the paths to your unzipped dataset folders.
# For example, if you placed them in subdirectories of your notebook: './FaceForensics++_data'
ff_local_dir = './FaceForensics++_data' # <--- UPDATE THIS PATH to your FaceForensics++ root folder
celeb_local_dir = './Celeb-DF-v2_data' # <--- UPDATE THIS PATH to your Celeb-DF-v2 root folder

# --- FaceForensics++ Dataset Setup ---
ff_subfolders = {
    'train': 'train',
    'validation': 'val',
    'test': 'test'
}

ff_datasets = get_dataset('FaceForensics++', ff_local_dir, ff_subfolders, sample_train=10000, sample_val=1000, sample_test=1000)
ff_train_dataset = ff_datasets.get('train')
ff_val_dataset = ff_datasets.get('validation')
ff_test_dataset = ff_datasets.get('test')

# --- Celeb-DF-v2 Dataset Setup ---
celeb_subfolders = {
    'train': 'train',
    'validation': 'val',
    'test': 'test'
}

celeb_datasets = get_dataset('Celeb-DF-v2', celeb_local_dir, celeb_subfolders, sample_train=5000, sample_val=500, sample_test=500)
celeb_train_dataset = celeb_datasets.get('train')
celeb_val_dataset = celeb_datasets.get('validation')
celeb_test_dataset = celeb_datasets.get('test')

# --- Cross-Dataset Training and Evaluation ---

# Unzip the model if it's not already extracted
if not os.path.exists(final_model_path):
    print(f"Model directory '{final_model_path}' not found. Attempting to unzip '{model_zip_filename}'")
    if os.path.exists(model_zip_filename):
        try:
            with zipfile.ZipFile(model_zip_filename, 'r') as zip_ref:
                zip_ref.extractall('.') # Extract to current directory
            print(f"Unzipping of '{model_zip_filename}' complete.")
        except zipfile.BadZipFile:
            print(f"Error: Unzipping did not create '{final_model_path}'. Please check the zip file.")
        except Exception as e:
            print(f"An error occurred during unzipping: {e}")
    else:
        print(f"Error: '{model_zip_filename}' not found. Please ensure the model zip file is in the current directory.")

# Load the ultimate model
print(f"\nLoading base model from {final_model_path} for cross-dataset experiments...")
base_vit_model = ViTForImageClassification.from_pretrained(
    final_model_path,
    num_labels=2,
    ignore_mismatched_sizes=True
).to(device)

```

```

# --- Experiment 1: Train on FaceForensics++, Test on Celeb-DF ---
print("\n===== EXPERIMENT 1: Train on FaceForensics++, Test on Celeb-DF =====")
model_ff_to_celebd = ViTForImageClassification.from_pretrained(
    final_model_path, num_labels=2, ignore_mismatched_sizes=True).to(device)

# Unfreeze all layers for full fine-tuning on the new dataset
for param in model_ff_to_celebd.parameters():
    param.requires_grad = True

if ff_train_dataset and ff_val_dataset and celeb_test_dataset:
    train_and_evaluate_model(
        model_to_train=model_ff_to_celebd,
        train_data=ff_train_dataset,
        val_data=ff_val_dataset,
        test_data=celeb_test_dataset,
        output_dir='./model_ff_trained_celebd_tested',
        patience=2
    )
else:
    print("Skipping Experiment 1 due to missing FaceForensics++ or Celeb-DF datasets.")

# --- Experiment 2: Train on Celeb-DF, Test on FaceForensics++ ---
print("\n===== EXPERIMENT 2: Train on Celeb-DF, Test on FaceForensics++ =====")
model_celebd_to_ff = ViTForImageClassification.from_pretrained(
    final_model_path, num_labels=2, ignore_mismatched_sizes=True).to(device)

# Unfreeze all layers for full fine-tuning on the new dataset
for param in model_celebd_to_ff.parameters():
    param.requires_grad = True

if celeb_train_dataset and celeb_val_dataset and ff_test_dataset:
    train_and_evaluate_model(
        model_to_train=model_celebd_to_ff,
        train_data=celeb_train_dataset,
        val_data=celeb_val_dataset,
        test_data=ff_test_dataset,
        output_dir='./model_celebd_trained_ff_tested',
        patience=2
    )
else:
    print("Skipping Experiment 2 due to missing Celeb-DF or FaceForensics++ datasets.")

```

```

import shutil
import os

# List of directories and files to clean up
cleanup_paths = [
    './vit_ultimate_checkpoints',
    './usmfe_vit_finetuned',
    './usmfe_vit_finetuned_high_acc',
    './usmfe_vit_90_percent_model_checkpoints',
    './usmfe_pillar1_vit_model.zip',
    './temp_ela.jpg'
]

# Add the dataset downloaded from kagglehub if it's not the primary dataset path
# The base_dataset_path '/content/140k-real-and-fake-faces/real_vs_fake/real-vs-fake/' is quite 1
# so I'll only suggest removing smaller, transient datasets if any.
# Assuming the main dataset is /content/140k-real-and-fake-faces/ and should be kept.

print("Starting cleanup of temporary files and old model checkpoints...")

```

```

for path_to_remove in cleanup_paths:
    if os.path.exists(path_to_remove):
        if os.path.isdir(path_to_remove):
            shutil.rmtree(path_to_remove)
            print(f"🗑️ Removed directory: {path_to_remove}")
        elif os.path.isfile(path_to_remove):
            os.remove(path_to_remove)
            print(f"🗑️ Removed file: {path_to_remove}")
    else:
        print(f"Skipping: {path_to_remove} (not found)")

print("🌟 Cleanup complete. Only the final model and necessary dataset files should remain.")

```

```

Starting cleanup of temporary files and old model checkpoints...
🗑️ Removed directory: ./vit_ultimate_checkpoints
Skipping: ./usmfe_vit_finetuned (not found)
🗑️ Removed directory: ./usmfe_vit_finetuned_high_acc
🗑️ Removed directory: ./usmfe_vit_90_percent_model_checkpoints
Skipping: ./usmfe_pillar1_vit_model.zip (not found)
Skipping: ./temp_ela.jpg (not found)
🌟 Cleanup complete. Only the final model and necessary dataset files should remain.

```

```

import os
import torch
import numpy as np
from PIL import Image
from torchvision import datasets, transforms
from transformers import (
    ViTImageProcessor,
    ViTForImageClassification,
    TrainingArguments,
    Trainer,
    EarlyStoppingCallback
)
from sklearn.metrics import accuracy_score, f1_score

# Set environment variable for better CUDA error reporting
os.environ["CUDA_LAUNCH_BLOCKING"] = "1"

# Set seeds for reproducibility
np.random.seed(42)
torch.manual_seed(42)

# --- 1. CONFIGURATION & PATHS ---
device = "cuda" if torch.cuda.is_available() else "cpu"
print(f"Using device: {device}")

# Existing model path from previous unzipped folder
model_load_path = "usmfe_vit_finetuned_high_acc"

# New dataset base path
# This is the root where 'train', 'valid', 'test' folders are located
base_dataset_path = "/content/140k-real-and-fake-faces/real_vs_fake/real-vs-fake/"

# Output directory for the continuously fine-tuned model
output_model_path = "./usmfe_vit_90_percent_model"

# Ensure the model directory exists by unzipping if necessary
if not os.path.exists(model_load_path):
    print(f"Model directory '{model_load_path}' not found. Attempting to unzip 'usmfe_pillar1_vit'")
    !unzip -q usmfe_pillar1_vit_model.zip
    if not os.path.exists(model_load_path):
        raise FileNotFoundError(f"Failed to find or unzip model directory: {model_load_path}")
    else:
        print(f"Successfully unzipped '{model_load_path}'.")

```

```

# Load the processor from the base model, as its config was not saved with the fine-tuned model.
# The processor's configuration typically doesn't change during fine-tuning.
processor = ViTImageProcessor.from_pretrained('google/vit-base-patch16-224')

# --- 2. DATASET PREPARATION & AUGMENTATION (REVISED) ---

# Ensure dataset is downloaded and unzipped if it doesn't exist
dataset_zip_name = '140k-real-and-fake-faces.zip'
dataset_kaggle_id = 'xhlulu/140k-real-and-fake-faces'
dataset_extract_dir = '/content/140k-real-and-fake-faces/' # Changed extraction directory to /cor

if not os.path.exists(base_dataset_path):
    print(f"Dataset directory '{base_dataset_path}' not found. Attempting to download and unzip '
    # Download the dataset if the zip file doesn't exist locally
    if not os.path.exists(dataset_zip_name):
        print(f"Downloading Kaggle dataset: {dataset_kaggle_id}")
        !kaggle datasets download -d {dataset_kaggle_id}

    # Unzip the dataset
    print(f"Unzipping '{dataset_zip_name}' to '{dataset_extract_dir}'...")
    !unzip -q {dataset_zip_name} -d {dataset_extract_dir}

    if not os.path.exists(base_dataset_path):
        raise FileNotFoundError(f"Failed to find or unzip dataset directory: {base_dataset_path}")
    else:
        print(f"Successfully prepared dataset at '{base_dataset_path}'.")

# Image transformations with Augmentation for training
train_transforms = transforms.Compose([
    transforms.RandomHorizontalFlip(p=0.5),
    transforms.RandomRotation(degrees=10),
    transforms.ColorJitter(brightness=0.1, contrast=0.1, saturation=0.1, hue=0.1),
    transforms.Resize((processor.size['height'], processor.size['width'])),
    transforms.ToTensor(),
    transforms.Normalize(mean=processor.image_mean, std=processor.image_std),
])

# Validation transformations (no aggressive augmentation)
val_transforms = transforms.Compose([
    transforms.Resize((processor.size['height'], processor.size['width'])),
    transforms.ToTensor(),
    transforms.Normalize(mean=processor.image_mean, std=processor.image_std),
])

class DatasetWrapper(torch.utils.data.Dataset):
    """Wrapper to apply transforms to a base dataset (e.g., Subset of ImageFolder)"""
    def __init__(self, base_dataset, transform=None):
        self.base_dataset = base_dataset
        self.transform = transform

    def __len__(self):
        return len(self.base_dataset)

    def __getitem__(self, idx):
        image, label = self.base_dataset[idx] # This returns PIL Image and int

        # Explicitly assert that labels are within the expected range for binary classification
        if not (0 <= label < 2):
            raise ValueError(f"Label {label} is out of expected range [0, 1] for binary classific

        if self.transform:
            image = self.transform(image)
        # Ensure labels are torch.long, as expected by CrossEntropyLoss
        return {"pixel_values": image, "labels": torch.tensor(label, dtype=torch.long)}

```

```

# Define paths to the 'train' and 'valid' subdirectories within the dataset
train_data_path = os.path.join(base_dataset_path, 'train')
val_data_path = os.path.join(base_dataset_path, 'valid')

# Load full raw datasets from the 'train' and 'valid' folders
# ImageFolder will now correctly assign 'fake' to 0 and 'real' to 1 for each
full_train_dataset_raw = datasets.ImageFolder(root=train_data_path)
full_val_dataset_raw = datasets.ImageFolder(root=val_data_path)

# Print ImageFolder class_to_idx mapping for debugging for both train and val
print(f"ImageFolder (Train) class_to_idx mapping: {full_train_dataset_raw.class_to_idx}")
print(f"ImageFolder (Validation) class_to_idx mapping: {full_val_dataset_raw.class_to_idx}")

# Randomly select a subset of 10,000 to 15,000 images in total, with an 80/20 train/validation split
# Target total images = 15,000. Train = 12,000, Val = 3,000.

# Determine the number of samples for training and validation subsets
target_total_images = 15000
target_train_size = int(0.8 * target_total_images)
target_val_size = target_total_images - target_train_size

# Adjust actual sample sizes to not exceed available images in respective datasets
actual_train_sample_size = min(target_train_size, len(full_train_dataset_raw))
actual_val_sample_size = min(target_val_size, len(full_val_dataset_raw))

# Randomly sample indices for the training and validation sets
train_indices = np.random.choice(len(full_train_dataset_raw), actual_train_sample_size, replace=False)
val_indices = np.random.choice(len(full_val_dataset_raw), actual_val_sample_size, replace=False)

# Create Subset objects from the raw datasets using the sampled indices
sampled_train_raw = torch.utils.data.Subset(full_train_dataset_raw, train_indices)
sampled_val_raw = torch.utils.data.Subset(full_val_dataset_raw, val_indices)

# Apply transformations using the DatasetWrapper
train_dataset = DatasetWrapper(sampled_train_raw, transform=train_transforms)
val_dataset = DatasetWrapper(sampled_val_raw, transform=val_transforms)

print(f"Training with {len(train_dataset)} images and validating with {len(val_dataset)} images.")

# --- 3. MODEL LOADING & LAYER MANAGEMENT ---

print(f"Loading model from: {model_load_path}")
model = ViTForImageClassification.from_pretrained(
    model_load_path,
    num_labels=2, # Explicitly setting num_labels to 2 for binary classification
    ignore_mismatched_sizes=True # Important for loading a fine-tuned model
).to(device)

# Freeze embedding and first 8 layers
for name, param in model.named_parameters():
    if "embeddings" in name:
        param.requires_grad = False
    elif "encoder.layer" in name:
        layer_num = int(name.split("encoder.layer.")[1].split(".")[0])
        if layer_num < 8:
            param.requires_grad = False
        else:
            param.requires_grad = True # Unfreeze layers 8-11
    elif "classifier" in name:
        param.requires_grad = True # Unfreeze the classifier
    else:
        param.requires_grad = False # Keep other parts frozen by default

trainable_params = sum(p.numel() for p in model.parameters() if p.requires_grad)

```

```

print(f"Trainable Parameters: {trainable_params:,}")

# --- 4. TRAINING ARGUMENTS ---
def compute_metrics(eval_pred):
    predictions, labels = eval_pred
    predictions = np.argmax(predictions, axis=1)
    accuracy = accuracy_score(labels, predictions)
    f1 = f1_score(labels, predictions, average='binary') # Assuming binary classification
    return {"accuracy": accuracy, "f1": f1}

training_args = TrainingArguments(
    output_dir=f"{output_model_path}_checkpoints", # Checkpoints will be saved here
    learning_rate=1e-5, # VERY LOW learning rate
    num_train_epochs=5,
    per_device_train_batch_size=32, # Higher batch size
    per_device_eval_batch_size=32,
    weight_decay=0.01,
    fp16=torch.cuda.is_available(),
    eval_strategy="epoch",
    save_strategy="epoch",
    load_best_model_at_end=True,
    metric_for_best_model="accuracy", # Target accuracy
    logging_dir='./logs',
    logging_steps=50,
    remove_unused_columns=False,
    report_to="none", # Prevents issues with default wandb logging if not configured
)

# --- 5. EXECUTION ---
trainer = Trainer(
    model=model,
    args=training_args,
    train_dataset=train_dataset,
    eval_dataset=val_dataset,
    compute_metrics=compute_metrics,
    callbacks=[EarlyStoppingCallback(early_stopping_patience=2)] # Optional: add early stopping
)

print("Starting Continuous Fine-Tuning...")
trainer.train()

# Save the final best model
model.save_pretrained(output_model_path)
processor.save_pretrained(output_model_path)
print(f"✅ Continuously fine-tuned model and processor saved to {output_model_path}")

# Verify saved model
if os.path.exists(output_model_path):
    print(f"Model directory contents: {os.listdir(output_model_path)}")
else:
    print(f"❌ Error: Final model directory was not created.")

```

```

Using device: cuda
ImageFolder (Train) class_to_idx mapping: {'fake': 0, 'real': 1}
ImageFolder (Validation) class_to_idx mapping: {'fake': 0, 'real': 1}
Training with 12000 images and validating with 3000 images.
Loading model from: usmfe_vit_finetuned_high_acc

Loading weights: 100% 200/200 [00:00<00:00, 5077.97it/s]

[transformers] `logging_dir` is deprecated and will be removed in v5.2. Please set `TensorBoardLogger._log_dir`
Trainable Parameters: 1,538
Starting Continuous Fine-Tuning...

```

 [1875/1875 16:58, Epoch 5/5]

Epoch	Training Loss	Validation Loss	Accuracy	F1
1	0.682746	0.654044	0.615667	0.560762
2	0.652945	0.629302	0.646667	0.598789
3	0.624077	0.616060	0.656000	0.611738
4	0.613383	0.609333	0.664667	0.625186
5	0.615001	0.607109	0.667000	0.629314

```

import os
import torch
import numpy as np
from PIL import Image
from torchvision import datasets, transforms
from transformers import (
    ViTImageProcessor,
    ViTForImageClassification,
    TrainingArguments,
    Trainer,
    EarlyStoppingCallback
)
from sklearn.metrics import accuracy_score, f1_score

# 1. SETUP & DEVICE
device = "cuda" if torch.cuda.is_available() else "cpu"
print(f"Using device: {device}")

# Existing model and dataset paths
model_load_path = "usmfe_vit_finetuned_high_acc"
base_dataset_path = "/content/140k-real-and-fake-faces/real_vs_fake/real-vs-fake/"
output_model_path = "./usmfe_vit_ultimate_90_model"

# Load processor and model
processor = ViTImageProcessor.from_pretrained('google/vit-base-patch16-224')
model = ViTForImageClassification.from_pretrained(
    model_load_path,

```